Prob 1: Design MDFA over $\Sigma = \{a, b\}$ such that every string must contain odd no. of occurrance of the substring "ab".

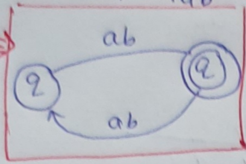
Sol: →

$|W|_x = r \pmod{n}$

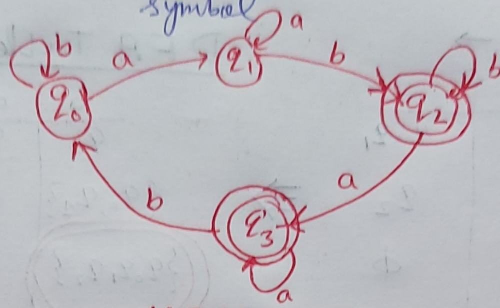
$|W|_{ab} = 1 \pmod{2}$

} Take ab as a single symbol. Construct DFA & Finally Break "ab" in two diff. symbol a & b

Idea ⇒



⇒



MDFA

Prob2: → How Many different DFA can be designed with fixed initial state over $\Sigma = \{a, b\}$ and Total No. of state is two (2).

Solution: → DFA = $\{Q, \Sigma, q_0, F, \delta\}$

- Q → $\{q_0, q_1\}$
- Σ → $\{a, b\}$
- q_0 → q_0 fixed
- F → [Final state may be 0, 1, 2]
- δ → $Q \times \Sigma = Q$

No Final state	One Final state	One Final state	two Final state
→ q_0 q_1	→ q_0 q_1	→ q_0 q_1	→ q_0 q_1
Total Possible DFA = 16	Total Possible DFA 16	Total Possible DFA → 16	Total Possible DFA → 16

All Possible Transition ↳

δ	a	b
q_0	q_0/q_1 (2)	q_0/q_1 (2)
q_1	q_0/q_1 (2)	q_0/q_1 (2)

$2 \times 2 \times 2 \times 2 = 16$

Answer

Total DFA = $16 + 16 + 16 + 16 = \boxed{64}$ different DFA

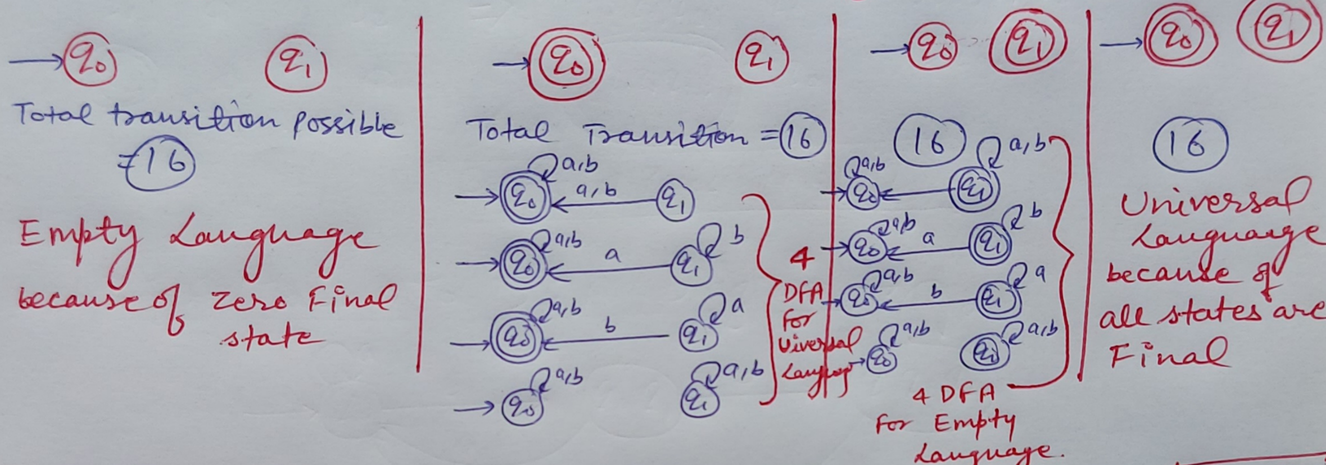
Prob 3:→ How many Different DFA is possible with fixed initial state over $\Sigma = \{a, b\}$ & Total No. of state is two (2) for

1. Accepting Universal Language.
2. Accepting Empty Language.

Solution:– DFA = $\{Q, \Sigma, q_0, F, \delta\}$

$Q \rightarrow \{q_0, q_1\}$, $\Sigma \rightarrow \{a, b\}$, $q_0 \rightarrow$ fixed, $F \rightarrow \{0, 1, 2\}$, $\delta \rightarrow Q \times \Sigma \rightarrow Q$

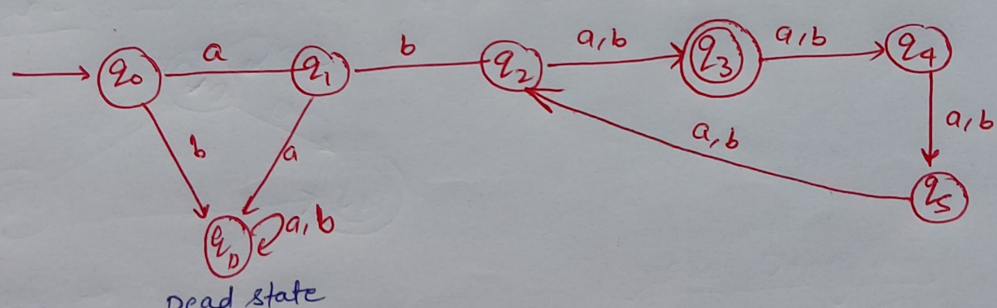
According to Final state total 4 category are possible



Answer ① ⟶ For Universal Language = $16 + 4 = \boxed{20 \text{ DFA}}$

② ⟶ For Empty Language = $16 + 4 = \boxed{20 \text{ DFA}}$

Problem:→ Design MDFA over $\Sigma = \{a, b\}$ such that it accepts all strings starting with "ab" and $|W| = 3 \pmod 4$.



Decision Property of F.A.

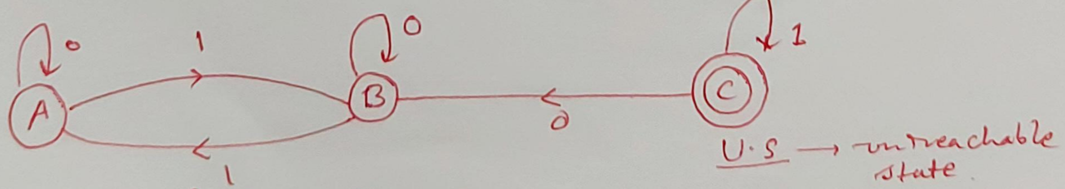
1. Emptyness or Non emptyness.
2. Finiteness or Non Finiteness.
3. Equalness.
4. Membership.

1. Emptyness Or Non-Emptyness :—

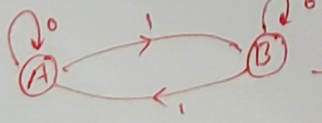
1. Select all the states that can not be reached from initial state (unreachable). And remove them.

2. If the resulting machine contain atleast one Final state then the FA Accept non-empty language.

3. If the resulting machine Free From Final state then the FA Accept empty language.



* By removing U.S. →



→ Empty language.

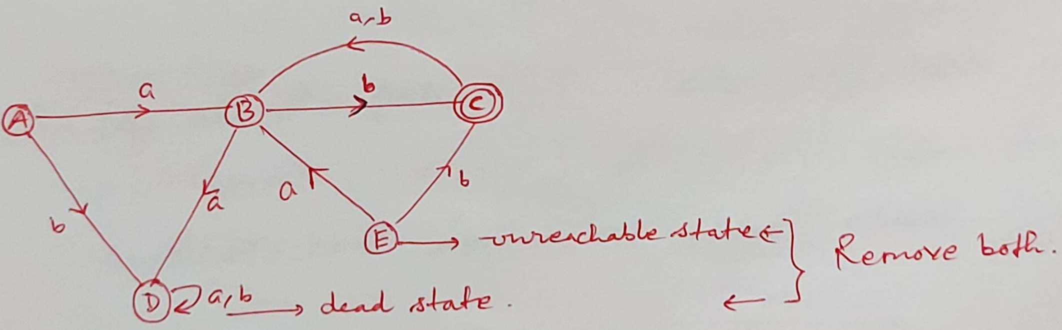
2. Finiteness or Infiniteness :—

1. Select all the state that can not be reached from initial state and remove them.

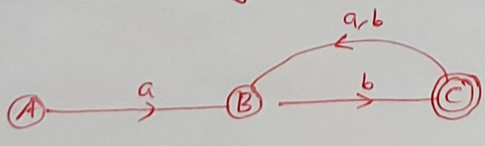
2. Select those states from which we can not reach on Final state (Dead state). and remove them.

3. If the resulting machine is Free from loop and cycle. then it Accept finite language.

4. If the Resulting Machine contains loop and cycle then it accept infinite language.



* After Removing D.S. & U.S. :—



Infinite Language.

v.imp

3. Equalness :—

1. two Finite state machine 'm_1' and 'm_2' are equal iff both of them Accept the same language.

Note :—

v.v imp 1. Equal Machine need not contain the same no. of states.

2. Two Machine 'm_1' & 'm_2' Isomorphic to eachother iff both of them Accept same language & have same No. of states.

3. If the machine are Isomorphic to eachother then they are equal. But equal Machine need not to be Isomorphic to each other.

Comparison Algorithm :— (For Equalness) :—

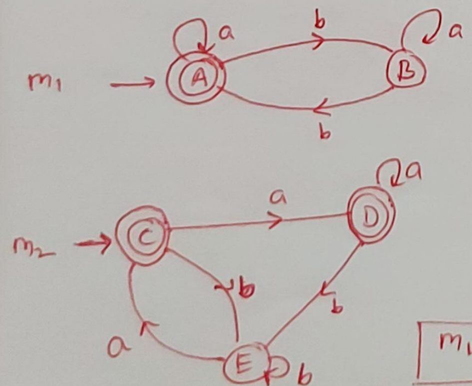
1. Let 'm_1' & 'm_2' be the two finite state machine, construct the transition table that contains pair of states (x, y) where 'x' $\in m_1$ & $y \in m_2$

2. Start the construction of Table with the pair of initial state. While constructing the table if we get any pair in the form of (final,— non-Final) or (Non Final,—Final) then stop the construction of Table & declare the two machine 'm_1' & 'm_2' are not equal.

3. Continue the construction of the Table for every new pair of the form (F, F) or (N.F., N.F.) & stop the construction whenever no new states occurs.

If all the pair of Transition ~~tat~~ table in the form (N.F., N.F.) or (F, F) then the two Machine 'm_1' & 'm_2' are equal.

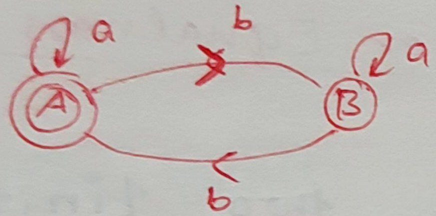
Example :—



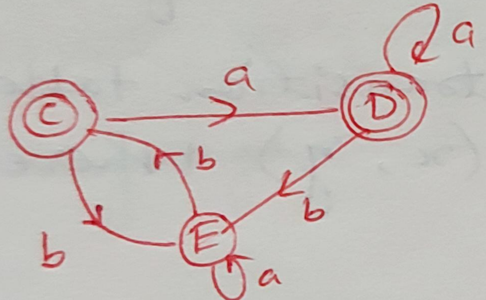
	a	b
A C	A D F F	B E NF NF
A D	A D F F	B E NF NF
B E	B C NF F	

$\boxed{m_1 \neq m_2}$

$m_1 \longrightarrow$



$m_2 \longrightarrow$



$$\boxed{m_1 = m_2}$$

	a	b
A C	A D F F	B E NF NF
A D	A D F F	B E NF NF
B E	B E NF NF	A C F F

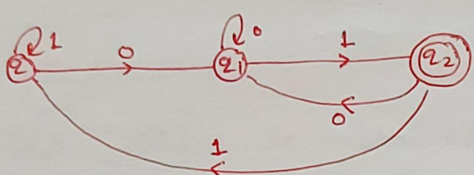
Membership :—

1. It is the property to verify the string accepted by F A or not.

2. The string 'w' is the member of F.A. if it is Accepted by F.A.

3. Draw the Transition path for 'w' and if the path ends on Final state then 'w' is a member otherwise 'w' is not a member.

Ex :—



w = 001 ⟶ Member.

w = 1011 ⟶ not a member.

Equal State :—

1. Two state 'p' and 'Q' are said to be equal if both 'p' and 'Q', $\delta(P, x)$, $\delta(Q, x)$ goes to final or non final for every input string 'x', $|x| \geq 0$

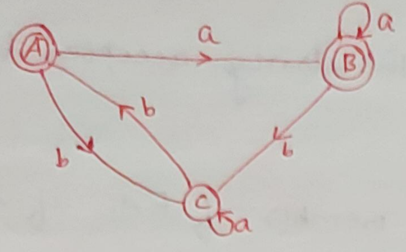
2. Both are goes to final state or both are goes to non- Final state.

3. $P = Q \Leftrightarrow$ $\delta(P, x)$ & $\delta(Q, x)$

non-Final state

Final state

Ex :-



$\delta(A,a) = B \rightarrow$ Final state
$\delta(B,a) = B \rightarrow$ Final state

$\delta(A,aab) = C \rightarrow$ Non - Final state
$\delta(B,aab) = C \rightarrow$ Non - Final state

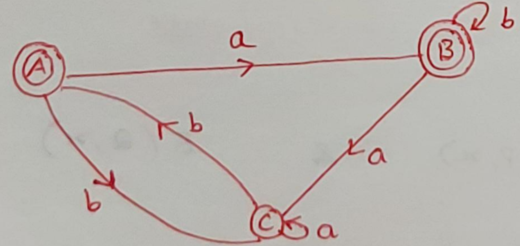
* **Note :-** After the conclusion the state 'A' and 'B' are equal, i.e. $\boxed{A = B}$

Unequal state :-

1. Two state 'P' and 'Q' are said to be unequal if $\delta(P,x)$ goes to Final state and $\delta(Q,x)$ goes to non-final state or vice-versa, for atleast one string 'x'.

2. $P \neq Q \iff \underset{\text{N.F.S (Non-Final state)}}{\overset{\text{F.S.}}{\delta(P,x)}} \;\&\; \underset{\text{F.S. (Final State)}}{\overset{\text{NFS}}{\delta(Q,x)}}$

Ex :-



$\delta(A,a) = B \longrightarrow$ F.S.
$\delta(B,a) = C \longrightarrow$ N.F.S.

$A \neq B$

* Note :—

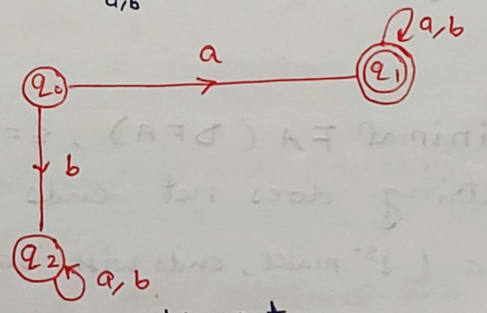
1. Every state is equal to itself.
2. Final state can never be equal to non final state. $\boxed{F.S \neq N.F.S.}$
3. Equality exist among the final states or non-final states.

Complement of F.A. :—

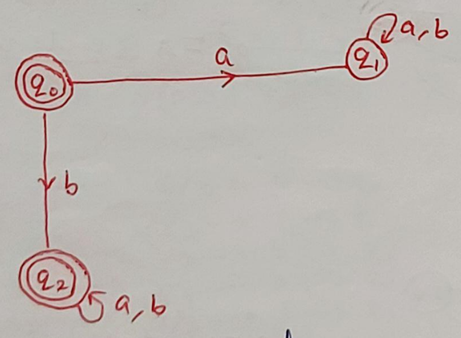
The FA which is obtained by interchanging Final & non-Final state is known as complement of F.A.

Ex :— $\Sigma = \{a, b\}$

let, $w = a\underset{a,b}{x}$



Before complement



After Complement

★ **Note :-**

1. $L(FA) \cap L(FA)^c = \phi$

★★ 2. $L(FA) \cup L(FA)^c = \Sigma^*$

3. No. of state in FA and in CFA (complemented -FA) is equal.

★★★ 4. Complement exist only for DFA and does not exist for NFA.

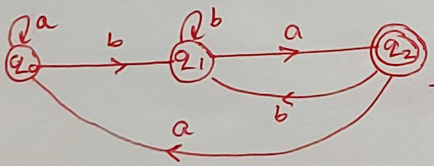
★★ 5. Dead state in FA is non- Final. but becomes final in complemented FA.

6. The unreachable state is still unreachable in both FA and CFA.

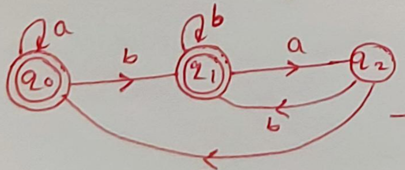
7. The unreachable state does not involve in the processing of any string both in FA and CFA.

Ex :- Construct the minimal FA (DFA), $\Sigma = \{a, b\}$ Where every string does not ends with 'a'.

Solution :- $w = x\,ba$ (1^{st} make, ends with 'ba' then complemented)



→ FA ends with 'ba'



→ CFA does not ends with 'ba'
↓
complemented FA.

EX :- Construct the minimal F.A. (DFA) that accept all the string of a's and b's where every string start with 'a' and length of string is not congruent to 2 (mod 3).

$q_2 \rightarrow$ Final state.

Solution :-

$2 \pmod 3 \rightarrow 0, 1, 2$

$q_0 \quad q_1 \quad q_2$

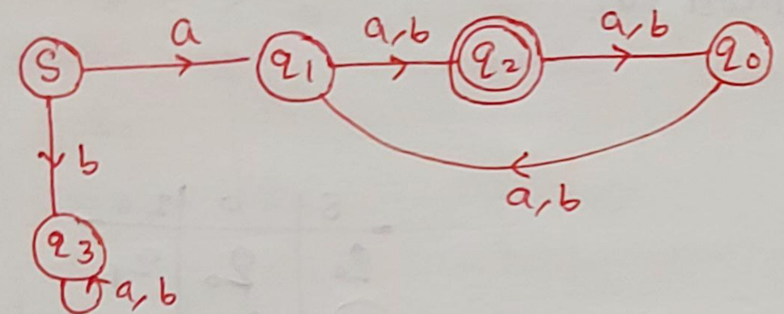
$w = a\,x$

$1/3 = 1 \rightarrow q_1$

$2/3 = 2 \rightarrow q_2$

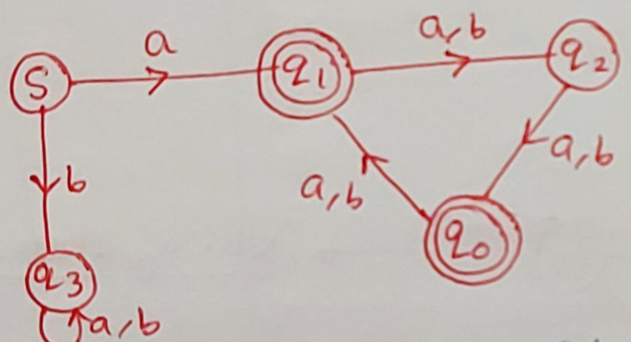
$3/3 = 0 \rightarrow q_0$

$= 2, 5, 8, 11, 14 \ldots$



dead state

F.A. Without complemented (congruent to 2 (mod 3))



complemented F.A. (does not congruent 2 (mod 3))

very imp

Conversion of NFA To DFA :-

The Process of Conversion of NFA to DFA is called subset construction.

$$\text{NFA} \Longrightarrow \text{DFA}$$

NFA $\Downarrow$ n state

DFA $\Downarrow$ m state

$$\boxed{1 \leq m \leq 2^n}$$

if $m = 1 \longrightarrow$ Best case

$1 < m < 2^n \longrightarrow$ Average case.

$m = 2^n \longrightarrow$ Worst case.

* Algorithm :-

$M = \{Q, \Sigma, \delta, F, q_0\} \longrightarrow$ NFA.

$M' = \{Q', \Sigma, \delta', F', q_0\} \longrightarrow$ DFA.

1. **Initial state :-** No change in initial state

2. **Construction of δ' :-**

Start the construction of δ' with initial state and continue for every new state and stop the construction where ever no new state occur.

3. **Final state(F)' :-** Every subset which contain the Final state of NFA is the Final state in DFA.

1. NFA Table to DFA Table :-

δ	a	b
$\rightarrow q_0$	q_0	q_0q_1
q_1	q_2	ϕ
q_2	ϕ	ϕ

NFA Table

δ'	a	b
$\rightarrow q_0$	q_0	q_0q_1
q_0q_1	q_0q_2	q_0q_1
q_0q_2	q_0	q_0q_1

DFA Table

$\delta(q_0q_1, a)$

$= \delta(q_0, a) \cup \delta(q_1, a)$

$= q_0 \cup q_2$

$= q_0q_2$

Calculations

2.

δ	a	b
$\rightarrow q_0$	$q_0 q_2$	q_1
q_1	q_2	$q_0 q_1$
(q_2)	q_0	ϕ

NFA Table

$\Leftrightarrow$

δ'	a	b
$\rightarrow q_0$	$q_0 q_2$	q_1
$q_0 q_2$	$q_0 q_2$	q_1
q_1	q_2	$q_0 q_1$
q_2	q_0	ϕ dead state
$q_0 q_1$	$q_0 q_2$	$q_0 q_1$
dead ϕ	dead ϕ	ϕ dead

DFA Table

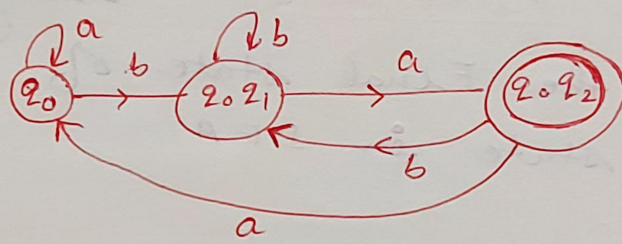
NFA Table to DFA diagram :-

①

δ	a	b
$\rightarrow q_0$	q_0	$q_0 q_1$
q_1	q_2	ϕ
(q_2)	ϕ	ϕ

NFA Table

$\Leftrightarrow$



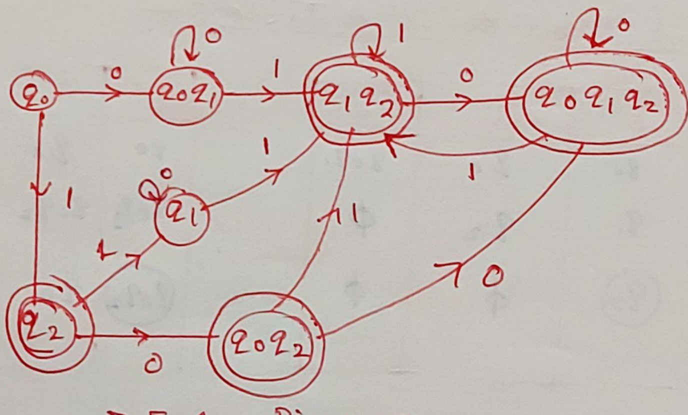
DFA diagram

②

δ	0	1
$\rightarrow q_0$	$q_0 q_1$	q_2
q_1	q_1	$q_1 q_2$
(q_2)	$q_0 q_2$	q_1

NFA Table

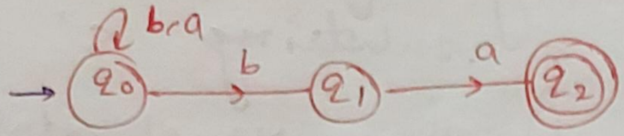
$\Leftrightarrow$



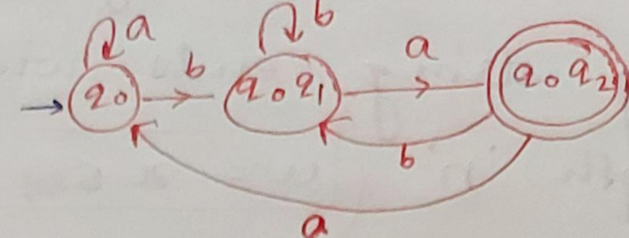
DFA Diagram

NFA Diagram to DFA diagram :-

①

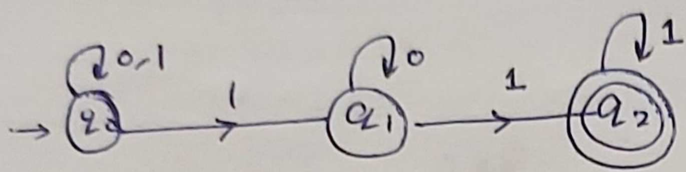


NFA diagram

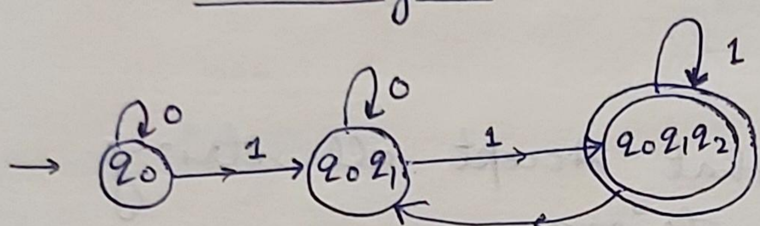


DFA diagram

②



NFA diagram

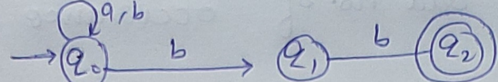


DFA diagram

Example 1: Conversion of NFA to DFA.

$\Sigma = \{a, b\}$

NFA :→



solution :→ Convert the NFA diagram to NFA Table

NFA Table →

δ	a	b
q_0	q_0	$q_0 q_1$
q_1	ϕ	q_2
q_2	ϕ	ϕ

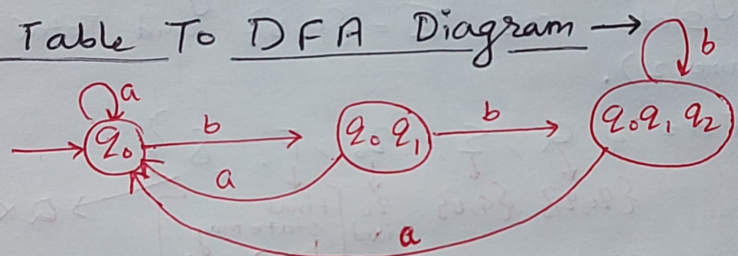
⟹

DFA Table →

δ	a	b
q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
$\{q_0 q_1 q_2\}$	q_0	$\{q_0 q_1 q_2\}$

Because of Presence of q_2

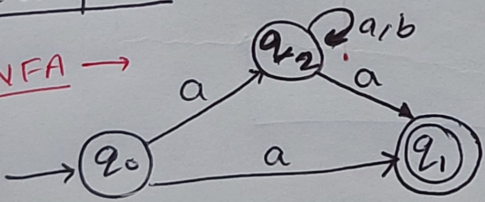
DFA Table To DFA Diagram →



- No change in Initial state
- change in Final state
-

Example 2 :→ Convert NFA to DFA.

NFA →



Solution :→

NFA Table

δ	a	b
q_0	q_1, q_2	ϕ
q_1	ϕ	ϕ
q_2	$q_2 q_1$	q_2

DFA Table

δ	a	b
q_0	$\{q_1 q_2\}$	$\{\phi\}$
⟹ $\{q_1 q_2\}$	$\{q_1 q_2\}$	$\{q_2\}$
$\{q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$\{\phi\}$	$\{\phi\}$	$\{\phi\}$

⟹

DFA Diagram